

东方资料包裹



English : README-en.pdf

日本語 : README-ja.pdf

中文 : README-zh.pdf

概述

东方资料包裹是一个数据包，可以让你获取有关东方Project中角色的信息。

涵盖了最新作品（锦上京）、旧作、书籍中的人物。

对于每个角色，例如灵梦，可以从源代码中轻松获取以下信息：

- 姓名
- 全名
- 每个角色都有独特的数字和字符
 - 它可用于 PlayerPrefs 和 switch 语句中的分支。

此外，姓名和全名被翻译成以下语言：

- 日语
 - 您还可以获得您的名字和全名的发音。
- 英语

- 中文

安装

导入 `TouhouDataPackage.unitypackage`。

要求

Unity 2020.2 或更高版本

此包使用 switch 表达式，因此需要支持 C# 8.0 或更高版本的 Unity 版本。

参考

所有这些包都存在于 `TouhouData` 命名空间中。

因此，使用此包时，请添加以下using指令。

```
using TouhouData;
```

Character

这是东方 Project 中的一个角色类别。

基本上，只存在静态变量和常量，但是类本身不是静态的，因此可以被继承。

Static 变量

姓名	解释
(任意角色)	关于东方 Project 人物的信息（只读）。 它可以以类似于枚举的方式被引用，例如 <code>Character.Reimu</code> 。 可以参考的字符请参阅下面的注释部分。

变量

姓名	解释
<code>Name</code>	东方角色的名字（只读）。不包括姓氏等。 返回的字符串将根据使用 <code>ChangeLanguage</code> 设置的语言而改变。
<code>FullName</code>	东方角色的全名（只读）。 返回的字符串将根据使用 <code>ChangeLanguage</code> 设置的语言而改变。
<code>Locales.(任何区域设置).Name</code>	东方角色的名字（只读）。不包括姓氏等。
<code>Locales.(任何区域设置).FullName</code>	东方角色的全名（只读）。
<code>Locales.Ja.NameKana</code>	东方角色的平假名名称（只读）。不包括姓氏等。 此变量仅存在于Locales.Ja中。
<code>Locales.Ja.FullNameKana</code>	这些是东方角色的平假名全名。此变量仅存在于Locales.Ja中。

姓名	解释
String	为每个东方角色分配一个唯一的 string 值（只读）。
ID	为每个东方角色分配一个唯一的 int 值（只读）。

Static 函数

姓名	解释
ChangeLanguage	设置Name和FullName的语言。
Get	这是一个返回特定Character的函数。int和string值可以指定为参数。特定のを返す関数です。

函数

姓名	解释
ToString	返回 String。

常量

姓名	解释
length	此包中包含的角色数（160）。
Strings.(任意角色)	任何东方角色的String。
IDs.(任意角色)	任何东方角色的ID。

课程

姓名	解释
Strings	这是一个拥有任意东方角色String作为成员的类别。
IDs	这是一个拥有任意东方角色ID作为成员的类别。
Locale	具有 Name 和 FullName 成员的类。
LocaleJa	从 Locale 继承的类，并具有附加的 NameKana 和 FullNameKana 成员。
CharacterLocales	一个具有成员 Ja (代表 LocaleJa)、 En 和 Zh (代表 Locale)的类。

(任意角色)

关于东方 Project 人物的信息（只读）。

它是一个静态的、只读变量，可以以类似于枚举的方式引用，例如 Character.Reimu。

可以参考的字符请参阅下面的注释部分。

```
Character character1 = Character.Reimu;  
Character character2 = Character.Marisa;
```

SelectedLanguage

Name 和 FullName 的 SystemLanguage（只读）。

该值可以是以下三个之一：

- SystemLanguage.Japanese
- SystemLanguage.English
- SystemLanguage.Chinese

Name

东方角色的名字（只读）。不包括姓氏等。

返回的字符串将根据使用 ChangeLanguage 设置的语言而改变。

```
Character character = Character.Reimu;  
Character.ChangeLanguage ("ja");  
Debug.Log (character.Name); // 霊夢  
Character.ChangeLanguage ("en");  
Debug.Log (character.Name); // Reimu  
Character.ChangeLanguage ("zh");  
Debug.Log (character.Name); // 灵梦
```

FullName

东方角色的全名（只读）。

返回的字符串将根据使用 ChangeLanguage 设置的语言而改变。

```
Character character = Character.Reisen;  
Character.ChangeLanguage (SystemLanguage.Japanese);  
Debug.Log (character.FullName); // 鈴仙・優曇華院・イナバ  
Character.ChangeLanguage (SystemLanguage.English);  
Debug.Log (character.FullName); // Reisen Udongein Inaba  
Character.ChangeLanguage (SystemLanguage.Chinese);  
Debug.Log (character.FullName); // 铃仙・优昙华院・因幡
```

Locales

翻译的信息存储在这个变量中。

语言和区域的对应关系如下：

- Locales.Ja：日语
- Locales.En：英语
- Locales.Zh：中文

Locales.(任何区域设置).Name

东方角色的名字（只读）。不包括姓氏等。

```
Character character = Character.Remilia;
Debug.Log (character.Locales.Ja.Name); // レミリア
Debug.Log (character.Locales.En.Name); // Remilia
Debug.Log (character.Locales.Zh.Name); // 蕾米莉亚
```

Locales.(任何区域设置).FullName

东方角色的全名（只读）。

```
Character character = Character.Marisa;
Debug.Log (character.Locales.Ja.FullName); // 霧雨 魔理沙
Debug.Log (character.Locales.En.FullName); // Marisa Kirisame
Debug.Log (character.Locales.Zh.FullName); // 雾雨 魔理沙
```

Locales.Ja.NameKana

东方角色的平假名名称（只读）。不包括姓氏等。此变量仅存在于Locales.Ja中。

```
Debug.Log (Character.Sakuya.Locales.Ja.NameKana); // さくや
Debug.Log (Character.Nitori.Locales.Ja.NameKana); // にとり
Debug.Log (Character.Alice.Locales.Ja.NameKana); // ありす
```

Locales.Ja.FullNameKana

这些是东方角色的平假名全名。此变量仅存在于Locales.Ja中。

```
Debug.Log (Character.Komachi.Locales.Ja.FullNameKana); // おのづか こまち
Debug.Log (Character.Tewi.Locales.Ja.FullNameKana); // いなば てゐ
Debug.Log (Character.Eiki.Locales.Ja.FullNameKana); // しき えいき・やまざなどう
```

String

分配给东方角色的唯一string值（只读）。

这个值和东方角色的静态变量名一模一样。

此外，`ToString` 返回该值。

此值在未来的更新中不会改变。

因此您可以安全地将该值用作 `PlayerPrefs` 键。

```
Debug.Log (Character.Reisen.String); // Reisen
Debug.Log (Character.ReisenSecond.String); // ReisenSecond
PlayerPrefs.SetString(Character.Reisen.String + "_Nickname", "优昙华");
PlayerPrefs.SetInt(Character.ReisenSecond.String + "_Power", 20);
```

ID

为每个东方角色分配一个唯一的 `int` 值（只读）。

该值与 `String` 按英文词典顺序排序时的索引相同。

此值将会在未来的更新中改变。

因此，不建议将其用作 `PlayerPrefs` 键。

```
Debug.Log (Character.Akyuu.ID); // 0
Debug.Log (Character.Alice.ID); // 1
Debug.Log (Character.Zanmu.ID); // 165
```

ChangeLanguage

设置 `Name` 和 `FullName` 的语言。

您可以将 `SystemLanguage` 和 `string` 作为参数传递。

`string` 不区分大小写。

如果您传递的参数所使用的语言不受此包支持，则该语言将为英语。

在游戏运行的第一个脚本中或设置语言的脚本中执行此功能。

如果不运行该函数，语言将根据 `Application.systemLanguage` 自动设置。

参数值和配置语言

- 参数：设置语言
- `SystemLanguage.Japanese`：日语
- `SystemLanguage.English`：英语
- `SystemLanguage.Chinese`, `SystemLanguage.ChineseSimplified`, `SystemLanguage.ChineseTraditional`：中文
- `"ja"`：日语
- `"en"`：英语
- `"zh"`：中文

- 其他：英语

```
Character.ChangeLanguage (SystemLanguage.Japanese); // 日语
Character.ChangeLanguage ("en"); // 英语
Character.ChangeLanguage ("Zh"); // 中文
Character.ChangeLanguage ("aaaaa"); // 英语
```

Get

这是一个返回特定Character的函数。int和string值可以指定为参数。

如果指定了 int 值，则返回 ID 与该 int 值匹配的东方角色。

给定一个string值，返回具有匹配的string和 **String** 的东方角色。

无论哪种情况，如果不存在这样的Character，则返回 null。

```
Character character1 = Character.Get (0);
Character character2 = Character.Get ("Reimu");
Character character3 = Character.Get (-999);
Character character4 = Character.Get ("aaaaaaaaaaaaaaaa");

if (character1 != null)
{
    Debug.Log (character1.FullName); // 稗田 阿求
}
Debug.Log (character2?.FullName); // 博丽 灵梦
Debug.Log (character3?.FullName); // null
Debug.Log (character4); // null
```

ToString

返回 **String**。

```
Debug.Log (Character.Reisen.ToString ()); // Reisen
Debug.Log (Character.ReisenSecond.ToString ()); // ReisenSecond
PlayerPrefs.SetString(Character.Reisen.ToString () + "_Nickname", "优昙华");
PlayerPrefs.SetInt(Character.ReisenSecond.ToString () + "_Power", 20);
```

length

此包中包含的角色数（160）。

当您想要获取一个随机字符或运行 for 循环时这很有用。

```
Character randomCharacter = Character.Get (Random.Range (0, Character.length)); //
随机角色

for (int i = 0; i < Character.length; i++)
{
    Character character = Character.Get (i);
    if (character != null)
    {
        Debug.Log (character.String); // Akyuu, Alice, Ariya, ... , Yuuma, Yuyuko,
Zanmu
    }
}
```

Strings.(任意角色)

这是任何东方角色的String（常数）。

因为它是一个常量，所以它可以在 switch 语句中使用，也可以作为 PlayerPrefs 中的键。

```
Character character = Character.Get (Random.Range (0, Character.length));

switch (character.String)
{
    case Character.Strings.Ekisyua:
    case Character.Strings.Genjii:
    case Character.Strings.Rinnosuke:
        Debug.Log (character.FullName + "是男性。");
        break;
    case Character.Strings.Singyoku:
        Debug.Log (character.FullName + "可以是男性或女性。");
        break;
    default:
        Debug.Log (character.FullName + "是女性。");
        break;
}
```

IDs.(任意角色)

任意东方角色ID（常量）。

因为它是一个常量，所以可以在 switch 语句中使用。

```
Character character = Character.Get (Random.Range (0, Character.length));

switch (character.ID)
{
    case Character.IDs.Ekisyua:
    case Character.IDs.Genjii:
```

```
case Character.IDs.Rinnosuke:
    Debug.Log (character.FullName + "是男性。");
    break;
case Character.IDs.Singyoku:
    Debug.Log (character.FullName + "可以是男性或女性。");
    break;
default:
    Debug.Log (character.FullName + "是女性。");
    break;
}
```

Strings

这是一个拥有任意东方角色String作为成员类别。

所有成员都是常量，但由于它不是静态类，因此可以被继承。

IDs

这是一个拥有任意东方角色ID作为成员类别。

所有成员都是常量，但由于它不是静态类，因此可以被继承。

Locale

具有 **Name** 和 **FullName** 成员的类。

LocaleJa

从 **Locale** 继承的类，并具有附加的 **NameKana** 和 **FullNameKana** 成员。

CharacterLocales

一个具有成员 **Ja** (代表 **LocaleJa**)、**En** 和 **Zh** (代表 **Locale**) 的类。

在(任意角色).Locales中使用此类。

示例

Samples 目录包含如何使用此包的示例，以及使用时有用的模板。

- **TouhouDataPackageSample.unity** : 这是使用 **Character** 创建的东方人物百科全书的样本。
 - **Scripts/TouhouDataPackageSample.cs** : 这是该场景使用的脚本。它使用了 **Character** 类的所有特性。
- **TouhouDataPackageExtendedSample.unity** : 这是扩展 **Character** 来创建东方人物百科全书的示例。
 - **Scripts/TouhouDataPackageExtendedSample.cs** : 这是该场景使用的脚本。我们使用一个继承了**Character**类的所有功能的类。
- **TouhouDataPackageTemplate.txt** : 所有字符的 switch 语句模板都写在这里。

如果你在 Assets/Resources/Pictures/Character 目录中准备一个与 Character.String 同名的图片（例如 Reimu.png），则该角色图像将显示在示例场景中。

评论

套装中包含的角色

以下是该包所支持的角色的英文名称。（如果静态变量名与英文名不同，则会写在括号中。）

- Akyuu
- Alice
- Ariya
- Aunn
- Aya
- Benben
- Biten
- Byakuren
- Chen
- Chimata
- Chimi
- Chiyari
- Chiyuri
- Cirno
- Clownpiece
- Daiyousei
- Doremy
- Eika
- Eiki
- Eirin
- Ekisya
- Elis
- Ellen
- Elly
- Enoko
- Eternitylarva
- Flandre
- Futo
- Gengetu
- Genjii
- Hatate
- Hecatia
- Hina
- Hisami
- Ichirin
- Iku
- Junko
- Jyoon

- Kagerou
- Kaguya
- Kana
- Kanako
- Kasen
- Keiki
- Keine
- Kikuri
- Kisume
- Koakuma
- Kogasa
- Koishi
- Kokoro
- Komachi
- Konngara
- Kosuzu
- Kotohime
- Kurumi
- Kutaka
- Kyouko
- Letty
- Lilywhite
- Luize
- Lunarchild
- Lunasa
- Lyrice
- Mai
- Mamizou
- Maribel
- Marisa
- Mayumi
- Medicine
- Megumu
- Meira
- Meirin
- Merlin
- Mike
- Miko
- Mima
- Minamitsu
- Minoriko
- Misumaru
- Miyoi
- Mizuchi
- Mokou
- Momizi

- Momoyo
- Mugetu
- Mystia
- Nareko
- Narumi
- Nazrin
- Nemuno
- Nina
- Nitori
- Nue
- Okina
- Orange
- Parsee
- Patchouli
- Raiko
- Ran
- Reimu
- Reisen
- ReisenSecond(冷仙)
- Remilia
- Renko
- Rika
- Rikako
- Rin
- Ringo
- Rinnosuke
- Rumia
- Ruukoto
- Sagume
- Saki
- Sakuya
- Sanae
- Sannyo
- Sara
- Sarial
- Satono
- Satori
- Seiga
- Seija
- Seiran
- Sekibanki
- Shinki
- Shinmyoumaru
- Shion
- Shizuha
- Singyoku

- Starsapphire
- Suika
- Sumireko
- Sunnymilk
- Suwako
- Syou
- Takane
- Teireida(舞)
- Tenshi
- Tewi
- Tojiko
- Tokiko
- Toyohime
- Tsukasa
- Ubame
- Urumi
- Utsuho
- Wakasagihime
- Wriggle
- Yachie
- Yamame
- Yatsunashi
- Yoriyime
- Yoshika
- Youmu
- Yuiman
- Yukari
- Yuki
- Yumeko
- Yumemi
- Yuugenmagan
- Yuugi
- Yuuka
- Yuuma
- Yuyuko
- Zanmu

未在上面列出的角色将不会包含在包中。

特别是，符合以下条件的角色将被排除在此包之外：

- 仅限设定中的角色
 - 冴月麟、岩笠、命莲 等。
- 一个非常次要的角色
 - 万岁乐、地精 等。
 - 包装内含 留琴、朱鹭子、易者
- 同人小说人物

- 濑笈 叶、左城宫 则纱 等。
- 非生物
 - 咪咪号、通往地灵殿的第一阶段的路上的岩石 等。
- 非东方角色
 - 西方和蓬莱少女绘幻想中的角色

若您想使用此包中未包含的角色，请参考 `Samples/Scripts/TouhouDataPackageExtendedSample.cs` 并创建一个扩展 `TouhouData.Character` 类的类。

考虑到将来的角色添加，不建议修改 `TouhouData.Character` 类本身。

关于名字

妖精的名字处理

妖精的名字，例如斯塔萨菲雅，通常由两个单词组成。

在这种情况下，两个词都被视为名字，没有姓氏。

此外，在英文名称中，两个单词之间没有空格，并且第二个单词不大写。

关于角色的英文名

原始符号将优先。

例如美铃的英文名 `Meirin`，一般拼音写为 `Meiling`，但本包使用了 `Meirin`，也就是原作的使用方式。

在日语中，`ji` 写成 `ji` 和 `zi`，而 `shi` 写成 `si` 和 `shi`，因此赫本拼写是混合的，但原始拼写优先，并且允许混合使用。我们正在做这件事。

作为例外，满足以下条件的字符将使用Touhou Wiki中的符号书写。

- 原文明显是错别字
 - 斯塔萨菲雅
- 原来的拼写使得正确发音困难。
 - 幽香
 - 勇仪
 - 八千慧
 - 磨弓
 - 千亦
 - 尤魔

关于角色的中文名

全部遵循THBWiki符号。

关于String

它与 `Locales.En.Name` 基本相同，但如果 `Locales.En.Name` 中有重复的字符，则会有不同的值。

关于summary

为了让summary更容易看清，原则上只实现日语和英语的summary。

参考

- [東方元ネタwiki 2nd](#)
- [Touhou Wiki](#)
 - [其他拼写](#)
- [THBWiki](#)

支持

如果您对此包有任何问题或意见，请在此处联系我们。

- 电子邮件：teracyans1223@gmail.com
 - 请使用@代表天才妖精。
- GitHub：<https://github.com/TD12734/TouhouDataPackage/issues>